

Festivāls

Naira atrodas festivālā un spēlē spēli, kur galvenā balva ir ceļojums uz Kolorada lagūnu. Spēlē, izmantojot žetonus, tiek pirkti kuponi. Pērkot kuponus, iespējams iegūt papildu žetonus. Mērķis ir iegūt pēc iespējas vairāk kuponus.

Viņa spēli sāk ar A žetoniem. Ir N kuponi, kas sanumurēti no 0 līdz $N - 1$. Nairai jāsamaksā $P[i]$ žetoni ($0 \leq i < N$), lai nopirktu i -to kuponu (un viņai pirms pirkuma jābūt vismaz $P[i]$ žetoniem). Viņa var nopirkt katru kuponu ne vairāk kā vienu reizi.

Turklāt, katram kuponam i ($0 \leq i < N$) ir **tips**, kas apzīmēts ar $T[i]$ – vesels skaitlis **starp 1 un 4, ieskaitot**. Pēc tam, kad Naira nopērk i -to kuponu, viņas atlikušo žetonu skaits tiek sareizināts ar $T[i]$. Formāli, ja viņai kādā spēles brīdī ir X žetoni un viņa nopērk i -to kuponu (lai to izdarītu, nepieciešams $X \geq P[i]$), tad viņai būs $(X - P[i]) \cdot T[i]$ žetoni pēc pirkuma.

Jūsu uzdevums ir noteikt, kurus kuponus un kādā secībā Nairai jānopērk, lai kopējais **kuponu** skaits, kas viņai ir beigās, būtu lielākais iespējamais. Ja ir vairāk nekā viena pirkumu secība, kas ļauj sasniegt mērķi, var atgriezt jebkuru no tām.

Implementācijas detaļas

Jums jāimplementē šāda procedūra.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : Nairai esošo žetonu skaits sākumā.
- P : masīvs garumā N , kurā norādītas kuponu cenas.
- T : masīvs garumā N , kurā norādīti kuponu tipi.
- Šī procedūra katram testam tiek izsaukta tieši vienreiz.

Procedūrai jāatgriež masīvs R , kurā Nairas pirkumi norādīti šādi:

- R garumam jābūt vienādam ar maksimālo kuponu, ko viņa var nopirkt, skaitu.
- Masīva elementi ir kuponu, kurus viņai jānopērk, indeksi hronoloģiskā secībā. Tas ir, viņa vispirms nopērk $R[0]$ -to kuponu, tad $R[1]$ -o kuponu, utt.
- Visiem R elementiem jābūt dažādiem.

Ja nevar nopirkt nevienu kuponu, R jābūt tukšam masīvam.

Ierobežojumi

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ katram i , kur $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ katram i , kur $0 \leq i < N$.

Apakšuzdevumi

Apakšuzdevums	Punkti	Papildu ierobežojumi
1	5	$T[i] = 1$ katram i , kur $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ katram i , kur $0 \leq i < N$.
3	12	$T[i] \leq 2$ katram i , kur $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Naira var nopirkt visus N kuponus (kaut kādā secībā).
6	16	$(A - P[i]) \cdot T[i] < A$ katram i , kur $0 \leq i < N$.
7	18	Bez papildu ierobežojumiem.

Piemēri

1. piemērs

Aplūkosim šādu izsaukumu.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Nairai sākumā ir $A = 13$ žetoni. Viņa var nopirkt 3 kuponus šādā secībā:

Nopirktais kupons	Kupona cena	Kupona tips	Žetonu skaits pēc pirkuma
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

Šajā piemērā Nairai nav iespējams nopirkt vairāk nekā 3 kuponus, un iepriekš aprakstītā pirkumu secība ir vienīgais veids, kā viņa var nopirkt visus 3 kuponus. Tātad, procedūrai jāatgriež $[2, 3, 0]$.

2. piemērs

Aplūkosim šādu izsaukumu.

```
max_coupons(9, [6, 5], [2, 3])
```

Šajā piemērā Naira var nopirkt abus kuponus jebkādā secībā. Tātad, procedūrai jāatgriež vai nu $[0, 1]$, vai $[1, 0]$.

3. piemērs

Aplūkosim šādu izsaukumu.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

Šajā piemērā Nairai ir viens žetons, kas nav pietiekami, lai nopirktu kaut vienu kuponu. Tātad, procedūrai jāatgriež $[]$ (tukšs masīvs).

Paraugvērtētājs

Ievaddatu formāts:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Izvaddatu formāts:

```
S
R[0] R[1] ... R[S-1]
```

Šeit S ir masīva R , ko atgriež `max_coupons`, garums.